

Содержание

Асимптотическая сложность алгоритмов и сортировки	1
0. Повторение: функция $T(n)$	1
1. Зачем вообще оценивать алгоритмы	1
2. Асимптотическая сложность и порядок роста	2
3. Классификация сложности алгоритмов	6
4. Оценка по памяти	6
5. Сортировки	7
Краткое резюме связей между понятиями	13
6. Мастер-теорема (Master Theorem)	13

Асимптотическая сложность алгоритмов и сортировки

0. Повторение: функция $T(n)$

На прошлой лекции была введена функция $T(n)$ — это **время работы алгоритма как функция от размера входных данных n** . Она показывает, сколько «элементарных операций» (шагов, сравнений, присваиваний и т.п.) выполнит алгоритм в зависимости от того, насколько велик вход. Именно $T(n)$ мы затем анализируем и оцениваем асимптотически — то есть смотрим не на точное количество операций, а на то, *как быстро* оно растёт при увеличении n . На прошлом занятии разбирались два примера вычисления $T(n)$ для конкретных алгоритмов (подсчёт количества операций по коду), которые служат отправной точкой для сегодняшней темы — перехода от точной функции $T(n)$ к её асимптотической оценке.

1. Зачем вообще оценивать алгоритмы

Прежде чем говорить про нотации, стоит понять, зачем это нужно:

- **Сравнение алгоритмов независимо от «железа» и языка программирования.** Один и тот же алгоритм на разных компьютерах и разных языках будет работать с разной скоростью в секундах, но *порядок роста* времени работы от размера входа — это свойство самого алгоритма, а не машины.
- **Прогноз масштабируемости.** Зная, что алгоритм работает за $O(n^2)$, мы можем заранее сказать: если увеличить вход в 10 раз, время работы увеличится примерно в 100 раз. Это критично, когда данные растут (базы данных, поисковые системы и т.д.).
- **Выбор алгоритма под задачу.** Если размер входа маленький, разница между $O(n^2)$ и $O(n \log n)$ может быть не важна. Но если вход — миллионы элементов, выбор алгоритма с худшим порядком роста может сделать программу практически неработоспособной.
- **Гарантии в худшем случае.** Асимптотические оценки часто нужны, чтобы дать *гарантию*: «алгоритм никогда не будет работать дольше, чем $c \cdot g(n)$ »

шагов», что важно в системах, где нужна предсказуемость (реальное время, критичные системы).

Таким образом, асимптотическая сложность — это способ описать *качественное* поведение алгоритма при росте входных данных, отбросив несущественные детали (константы, особенности конкретной реализации).

2. Асимптотическая сложность и порядок роста

Порядок роста (order of growth) — это то, как быстро растёт функция $T(n)$ при $n \rightarrow \infty$, без учёта конкретных констант и «мелких» слагаемых, которые становятся незначительными на больших n .

Чтобы формально описывать порядок роста, используют так называемые **асимптотические нотации** — в первую очередь O -нотацию.

Замечание про происхождение обозначения. Буква O происходит от немецкого слова *Ordnung* — «порядок». То есть $O(g(n))$ дословно означает «порядок $g(n)$ », то есть класс функций, растущих не быстрее, чем $g(n)$ (с точностью до константы).

2.1. Аналогия с асимптотами

В лекции проводится аналогия: **максимальная и минимальная оценки сложности похожи на асимптоты функции.**

Вспомним, что такое асимптота из математического анализа: это прямая (или кривая), к которой график функции приближается сколь угодно близко, но обычно не пересекает (или пересекает лишь конечное число раз) при стремлении аргумента к бесконечности. Асимптота *ограничивает* поведение функции на бесконечности сверху или снизу.

Точно так же:

- функция $g(n)$ в оценке $O(g(n))$ — это **верхняя граница**, к которой (с точностью до множителя) «прижимается» реальное время работы $T(n)$ сверху при больших n ;
- аналогично для нижней оценки существует функция, которая ограничивает $T(n)$ снизу.

То есть вместо того, чтобы точно вычислять $T(n)$, мы находим более простую функцию $g(n)$ (например, n^2 , $n \log n$, 2^n), которая «обрамляет» рост $T(n)$ сверху и/или снизу, подобно тому как асимптота обрамляет поведение кривой.

2.2. O -нотация — верхняя асимптотическая оценка

Определение. Говорят, что $f(n) = O(g(n))$, если

$$\exists c > 0, \exists n_0 > 0 : \forall n > n_0 \quad 0 \leq f(n) \leq c \cdot g(n).$$

Разберём это определение по частям, «переводя» кванторы на обычный язык:

- $\exists c > 0$ — **существует какая-то положительная константа c** (она может быть любой: 2, 100, 0.001 — главное, что она *фиксирована* и не зависит от n). Эта константа нужна, чтобы «не придирались» к постоянным множителям: $f(n) = 5n$ и $f(n) = 1000n$ — это, с точки зрения O -нотации, «одно и то же» — обе будут $O(n)$.
- $\exists n_0 > 0$ — **существует некоторый порог по n** . Нас не интересует, что происходит при маленьких n (там могут быть любые особенности, «накладные расходы» и т.п.) — важно только поведение начиная с некоторого достаточно большого n_0 .
- $\forall n > n_0$ — **для всех n , больших этого порога**, неравенство должно выполняться *всегда*, без исключений.
- $0 \leq f(n) \leq c \cdot g(n)$ — собственно **сама оценка сверху**: значение функции $f(n)$ не превышает c умножить на $g(n)$.

Смысл простыми словами: $f(n) = O(g(n))$ означает, что начиная с некоторого n , функция f растёт **не быстрее**, чем g с точностью до постоянного множителя. $O(g(n))$ — это, по сути, *множество всех функций*, которые ведут себя «не хуже», чем $g(n)$ при больших n . Поэтому строго говорить нужно $f(n) \in O(g(n))$, хотя по традиции чаще пишут $f(n) = O(g(n))$.

Важное замечание, где легко ошибиться: $O(g(n))$ даёт оценку **сверху**. Запись $f(n) = O(n^2)$ не означает, что f растёт *ровно как* n^2 — она может расти и медленнее (например, $f(n) = n$ тоже удовлетворяет $f(n) = O(n^2)$, потому что «не быстрее n^2 » — верно и для линейной функции). Часто в разговорной речи O используют неформально как «оценку роста именно такого порядка», но формально это только верхняя граница.

2.3. Другие оценки: Ω и Θ

Помимо верхней оценки O , существуют ещё две ключевые нотации (в лекции они упомянуты как «две другие оценки»):

Ω -нотация — нижняя асимптотическая оценка.

$$f(n) = \Omega(g(n)) \iff \exists c > 0, \exists n_0 > 0 : \forall n > n_0 \quad 0 \leq c \cdot g(n) \leq f(n).$$

Здесь всё симметрично определению O , только неравенство «перевернуто»: $f(n)$ теперь **не меньше**, чем $c \cdot g(n)$, начиная с некоторого n_0 . Это означает, что f растёт **не медленнее**, чем g . Ω даёт гарантию «снизу» — алгоритм не может выполняться быстрее этого порядка.

Θ -нотация — точная (двусторонняя) асимптотическая оценка.

$$f(n) = \Theta(g(n)) \iff \exists c_1 > 0, c_2 > 0, n_0 > 0 : \forall n > n_0 \quad 0 \leq c_1 \cdot g(n) \leq f(n) \leq c_2 \cdot g(n).$$

Это означает, что $f(n)$ зажата между двумя константными кратными $g(n)$ — то есть f растёт **ровно с тем же порядком**, что и g (не быстрее и не медленнее, с точностью до констант).

Связь между нотациями, отмеченная в лекции:

$$f(n) = \Theta(g(n)) \implies f(n) = O(g(n)) \text{ и одновременно } f(n) = \Omega(g(n)).$$

Логика простая: если функция зажата между $c_1g(n)$ и $c_2g(n)$ (это и есть Θ), то в частности она удовлетворяет и просто верхней оценке (с константой c_2), и просто нижней оценке (с константой c_1). То есть Θ — **это самая сильная и самая информативная оценка**: она объединяет в себе и O , и Ω одновременно. Если про алгоритм известно только O , мы знаем лишь «не хуже чем», если известно только Ω — «не лучше чем», а если известно Θ — мы точно знаем порядок роста.

2.4. Не путать асимптотические оценки с лучшим/худшим/средним случаем!

Это очень частое и важное место, где путаются два **разных, независимых** понятия:

1. O , Ω , Θ — описывают **вид (форму) роста функции** $T(n)$: насколько быстро время работы растёт с ростом n (линейно, квадратично, логарифмически и т.д.). Это чисто математическое свойство функции.
2. **Лучший, худший и средний случаи (best/worst/average case)** — описывают, **при каких входных данных** мы измеряем время работы. Один и тот же алгоритм на входах одного размера n может работать по-разному в зависимости от *конкретного содержания* этих данных (например, отсортирован ли массив заранее).

Эти два понятия **ортогональны** друг другу — их можно комбинировать. Например, про алгоритм можно сказать:

- «сложность в худшем случае — $O(n^2)$ » — то есть для *самых неудобных* входных данных размера n время работы растёт не быстрее n^2 ;
- «сложность в лучшем случае — $O(n)$ » — то есть для *самых удобных* входных данных время растёт не быстрее n .

То есть у **одного и того же алгоритма** может быть несколько разных функций $T(n)$ — по одной на каждый случай (лучший/худший/средний), и у *каждой* из этих функций отдельно можно ставить оценку O , Ω или Θ . Ошибка, которую часто допускают: путать “ O ” с “худшим случаем” — они не синонимы! O -оценку можно ставить и для лучшего, и для среднего случая тоже (просто с другой функцией $g(n)$ внутри).

2.5. Арифметические свойства асимптотических оценок

Эти свойства позволяют оценивать сложность составных алгоритмов (когда, например, один блок кода идёт за другим, или один вызывается внутри другого),

не расписывая всё заново по определению.

1) Сложение функций:

$$f_1(n) + f_2(n) = O(\max(g_1(n), g_2(n))), \quad \text{если } f_1 = O(g_1), f_2 = O(g_2).$$

Смысл: если у нас есть два последовательных блока кода со сложностями g_1 и g_2 (например, сначала цикл на n шагов, потом цикл на n^2 шагов), то суммарная сложность определяется **более “тяжёлым” слагаемым** — растущим быстрее. Например, $O(n) + O(n^2) = O(n^2)$, потому что при больших n квадратичное слагаемое “забывает” линейное.

2) Умножение функций:

$$f_1(n) \cdot f_2(n) = O(g_1(n) \cdot g_2(n)).$$

Смысл: это правило работает, например, для **вложенных циклов**: если внешний цикл выполняется $O(g_1(n))$ раз, а внутренний — $O(g_2(n))$ раз, суммарная сложность — произведение. Классический пример: два вложенных цикла по n дают $O(n) \cdot O(n) = O(n^2)$.

3) Умножение на константу:

$$c \cdot f_1(n) = O(g_1(n)), \quad \text{если } f_1 = O(g_1).$$

Смысл: умножение функции на постоянный коэффициент **не меняет** асимптотическую оценку — ведь константы и так “прячутся” внутри константы c из определения O . Например, $3n = O(n)$, $1000n = O(n)$ — асимптотически это одно и то же.

4) Прибавление константы:

$$f_1(n) + c = O(g_1(n)), \quad \text{если } f_1 = O(g_1) \text{ (и } g_1(n) \not\rightarrow 0 \text{ или растёт)}.$$

Смысл: добавление константного слагаемого (например, “+5 операций на инициализацию”) тоже не влияет на асимптотику при больших n , потому что константа становится пренебрежимо мала по сравнению с растущей функцией $g_1(n)$.

Замечание. Все эти правила — не просто удобные “трюки”, а прямое следствие определения O через кванторы $\exists c, n_0$: константы и меньшие слагаемые всегда можно “поглотить”, подобрав достаточно большую константу c в самом определении.

Связь между Θ , O и Ω ещё раз, в виде формулы из лекции:

$\Theta(g(n)) \implies O(g(n)) = \Omega(g(n))$ (то есть верхняя и нижняя оценки “сходятся” на одной и той же функции).

3. Классификация сложности алгоритмов

Функции $g(n)$, которые чаще всего встречаются в оценках, принято упорядочивать по скорости роста от самой медленной к самой быстрой. Понимание этой иерархии важно, чтобы сразу “на глаз” оценивать, насколько хорош или плох алгоритм.

От медленного роста к быстрому (при $n \rightarrow \infty$):

$$O(1) \ll O(\log n) \ll O(n) \ll O(n \log n) \ll O(n^2) \ll O(n^3) \ll \dots \ll O(2^n) \ll O(n!)$$

- $O(1)$ — **константная сложность**. Время работы не зависит от размера входа (например, обращение к элементу массива по индексу).
- $O(\log n)$ — **логарифмическая сложность**. Типично для алгоритмов, которые на каждом шаге отбрасывают часть данных (например, бинарный поиск: на каждом шаге область поиска уменьшается вдвое).
- $O(n)$ — **линейная сложность**. Время растёт пропорционально размеру входа (например, один проход по всем элементам массива).
- $O(n \log n)$ — **линейно-логарифмическая сложность**. Характерна для эффективных алгоритмов сортировки (в частности, сортировки слиянием — см. ниже).
- $O(n^2), O(n^3), \dots$ — **полиномиальная сложность (в общем виде $O(n^k)$ для константы k)**. Типична для “наивных” алгоритмов с вложенными циклами (например, сортировка вставками в худшем случае — $O(n^2)$).
- $O(2^n)$ — **экспоненциальная сложность**. Время растёт как показательная функция от n — характерно для полного перебора всех подмножеств множества из n элементов. Такие алгоритмы становятся практически неприменимыми уже при относительно небольших n (десятки элементов).
- $O(n!)$ — **факториальная сложность**. Ещё быстрее растёт, чем экспоненциальная — типична для полного перебора всех перестановок n элементов (например, “наивное” решение задачи коммивояжёра). Уже при $n \approx 20$ -25 такой алгоритм невозможно выполнить за разумное время.

Важное замечание: алгоритмы с полиномиальной сложностью ($O(n^k)$ для фиксированного k) считаются “практически применимыми”, а с экспоненциальной или факториальной — как правило, нет (за исключением очень маленьких n). Это фундаментальное деление лежит в основе теории сложности вычислений (классы P и NP, которые здесь не рассматриваются подробно, но полезно иметь в виду контекст).

4. Оценка по памяти

Аналогично тому, как мы оцениваем **время работы** алгоритма функцией $T(n)$ и её асимптотикой, точно так же можно оценивать **объём используемой памяти** — то есть сколько дополнительных ячеек памяти (помимо самого входа) требуется алгоритму для работы, в зависимости от размера входа n .

Это называется **пространственной сложностью (space complexity)**, и к ней применяются **те же самые нотации** O , Ω , Θ — только вместо функции времени $T(n)$ рассматривается функция объема памяти $S(n)$.

Например:

- Алгоритм, которому нужно всего несколько дополнительных переменных независимо от размера входа, имеет память $O(1)$ (“работает на месте”, *in-place*).
- Алгоритм, которому требуется создать вспомогательный массив того же размера, что и вход, имеет память $O(n)$.

Это важно учитывать наравне со временем работы: иногда более быстрый по времени алгоритм требует значительно больше памяти (пример — сортировка слиянием, которая требует $O(n)$ дополнительной памяти, в отличие от сортировки вставками, которой достаточно $O(1)$ дополнительной памяти).

5. Сортировки

5.1. Ключ сортировки

Ключ сортировки (sort key) — это та характеристика (поле, атрибут) элемента, по значению которой производится сравнение и упорядочивание элементов. У одного и того же набора данных может быть несколько разных полей, но сортировка всегда производится **по какому-то одному выбранному ключу** (либо по комбинации ключей, если сортировка многоключевая).

Например, если у нас есть список студентов, у каждого из которых есть *возраст* и *оценка*, то мы можем: - отсортировать список **по ключу “возраст”** — тогда сравниваются только значения возраста; - либо отсортировать **по ключу “оценка”** — тогда сравниваются только оценки.

Остальные поля (не являющиеся ключом) при этом “едут вместе” с элементом, но не участвуют в сравнении.

5.2. Устойчивые сортировки (stable sorts)

Определение. Сортировка называется **устойчивой (stable)**, если она сохраняет **относительный порядок** элементов с **одинаковыми значениями ключа**, в котором они находились во входных данных.

То есть если в исходном массиве элемент A стоял раньше элемента B , и у A и B одинаковый ключ сортировки, то после устойчивой сортировки A по-прежнему будет стоять раньше B .

Почему это важно — пример со студентами (возраст и оценки).

Представим список студентов вида (Имя, Возраст, Оценка), уже отсортированный по оценке:

Имя	Возраст	Оценка
Аня	20	5
Боря	20	4
Витя	19	3

Теперь мы хотим отсортировать этот список **по возрасту**. У Ани и Бори одинаковый возраст (20), но разные оценки, и в исходном порядке Аня идёт раньше Бори.

- **Если сортировка устойчива**, то после сортировки по возрасту порядок будет: Витя (19), затем Аня (20), затем Боря (20) — то есть внутри группы с одинаковым возрастом (Аня и Боря) сохранится их прежний относительный порядок (Аня всё ещё перед Борей), унаследованный от предыдущей сортировки по оценке.
- **Если сортировка неустойчива**, то внутри группы “возраст = 20” порядок Ани и Бори может как сохраниться, так и “перевернуться” — это уже не гарантируется и зависит от конкретной реализации алгоритма.

Именно это имеется в виду под фразой из конспекта: «если сортировка не устойчивая, при двух сортировках получим разный результат». Смысл в следующем: если запускать неустойчивую сортировку на одинаковых по ключу элементах, порядок между ними не гарантирован — то есть, применяя сортировку последовательно по нескольким ключам (сначала по одному полю, потом по другому, как в примере выше), **устойчивость критически необходима**, чтобы результат многоключевой сортировки был предсказуемым и корректным. Без устойчивости “накопленный” порядок от предыдущей сортировки может быть случайным образом разрушен на следующем шаге.

Замечание. Устойчивость — это свойство *конкретного алгоритма сортировки* (и даже конкретной его реализации), а не общего “типа” задачи. Например, сортировка вставками и сортировка слиянием (в стандартной реализации) — устойчивы, а вот, например, быстрая сортировка (quicksort) в типичной реализации — неустойчива.

5.3. Сортировка вставками (Insertion Sort)

Идея: массив мысленно делится на две части — уже отсортированную (в начале) и ещё не обработанную (в конце). На каждом шаге берём первый элемент из необработанной части и **вставляем** его на нужное место в отсортированной части, сдвигая при необходимости большие элементы вправо.

Псевдокод:

```
insertionSort(a, n):
  for i = 1 to n-1:
    key = a[i]
    j = i - 1
    while j >= 0 and a[j] > key:
      a[j+1] = a[j]
```


$$j = j - 1$$

$$a[j+1] = \text{key}$$

Разбор пошагово:

1. **for i = 1 to n-1** — мы начинаем со второго элемента (индекс 1), так как один-единственный элемент $a[0]$ уже “тривиально отсортирован” сам по себе. К моменту, когда мы обрабатываем индекс i , часть массива $a[0..i-1]$ уже гарантированно отсортирована — это **инвариант** алгоритма, который выполняется на каждой итерации.
2. **key = a[i]** — запоминаем значение текущего элемента, который нужно вставить в отсортированную часть. Мы должны сохранить его в отдельную переменную, потому что при сдвиге элементов место $a[i]$ может быть перезаписано.
3. **j = i - 1** — устанавливаем указатель j на последний элемент уже отсортированной части — именно отсюда начнём сравнение справа налево.
4. **while j >= 0 and a[j] > key** — пока не дошли до начала массива **и** пока текущий элемент отсортированной части больше, чем key , значит key должен стоять левее этого элемента.
5. **a[j+1] = a[j]; j = j - 1** — сдвигаем найденный больший элемент на одну позицию вправо (освобождая место для key) и переходим к сравнению со следующим (более левым) элементом.
6. **a[j+1] = key** — как только цикл **while** завершился (либо дошли до начала массива, либо нашли элемент не больше key), это значит, что позиция $j+1$ — правильное место для key , и мы его туда записываем.

Наглядно: это очень похоже на то, как человек сортирует карты в руке — берёт следующую карту и вставляет её в нужное место среди уже разложенных по возрастанию карт.

Оценка сложности:

- **Лучший случай ($O(n)$):** массив уже отсортирован по возрастанию. Тогда для каждого i условие $a[j] > \text{key}$ сразу ложно (внутренний **while** не выполняется вообще), и на каждый элемент тратится $O(1)$ операций $\rightarrow$ суммарно $O(n)$.
- **Худший случай ($O(n^2)$):** массив отсортирован в **обратном** порядке (по убыванию). Тогда для каждого i элемент key нужно “протащить” через всю уже отсортированную часть — то есть внутренний цикл делает порядка i шагов, а суммарно по всем i от 1 до $n - 1$ получаем $1 + 2 + \dots + (n - 1) = \frac{n(n-1)}{2} = O(n^2)$.
- **Средний случай ($O(n^2)$):** для случайно перемешанного массива в среднем каждый элемент нужно сдвинуть примерно на половину пройденной части — то есть асимптотически (с точностью до постоянного множителя $\frac{1}{2}$) получаем всё тот же порядок роста $O(n^2)$.

Дополнительно (память): сортировка вставками работает “**на месте**” (**in-place**), то есть требует лишь $O(1)$ дополнительной памяти (одна переменная key и пара индексов) — в отличие от сортировки слиянием (см. ниже).

Также стоит отметить: сортировка вставками **устойчива** — при сдвиге элементов мы сдвигаем только те, что строго больше key ($a[j] > \text{key}$, а не $\geq$), поэтому равные key элементы никогда не “перепрыгиваются”, и их относительный порядок сохраняется.

5.4. Метод декомпозиции: «разделяй и властвуй» (Divide and Conquer)

Прежде чем переходить к сортировке слиянием, нужно понять общий приём, на котором она основана — метод «**разделяй и властвуй**» (декомпозиция задачи). Этот метод состоит из трёх шагов:

1. **Разделяй (Divide)** — разбить исходную задачу на несколько **более мелких подзадач того же типа** (обычно — примерно равного размера).
2. **Властвуй (Conquer)** — решить каждую подзадачу **рекурсивно** (то есть тем же самым методом), пока подзадача не станет настолько маленькой, что её решение тривиально (базовый случай рекурсии).
3. **Объединяй (Combine)** — скомбинировать решения подзадач в решение исходной, большой задачи.

Именно по этой схеме построена сортировка слиянием: массив делится пополам (разделяй), каждая половина сортируется рекурсивно (властвуй), а затем две отсортированные половины сливаются в одну (объединяй).

5.5. Сортировка слиянием (Merge Sort)

Идея: рекурсивно разбивать массив пополам до тех пор, пока не останутся тривиальные (из одного или нуля элементов) подмассивы, а затем последовательно **сливать** пары уже отсортированных подмассивов в один отсортированный массив бóльшего размера.

Разбор и исправление псевдокода В исходном виде псевдокод выглядел так:

```
mergeSort(a, l, r){  
  if l+1=r return;  
  mergeSort(a, l, (l+r)/2);  
  mergeSort(a, (l+r)/2, r);  
  Merge(a, l, r);  
}
```

Здесь используется соглашение о **полуоткрытом диапазоне индексов** $[l, r)$ — то есть функция $\text{mergeSort}(a, l, r)$ сортирует элементы массива с индекса l **включительно** по индекс r **не включительно** (иными словами, элементы $a[l]$, $a[l+1]$, ..., $a[r-1]$). Это важно проговорить явно, иначе непонятно, почему обе рекурсивные вызовы используют одну и ту же середину mid без сдвига $+1$.

При таком соглашении: - левая половина — это $[l, mid)$, то есть `mergeSort(a, l, mid)`; - правая половина — это $[mid, r)$, то есть `mergeSort(a, mid, r)`; - индекс `mid` не “дублируется” между половинами, потому что правая граница каждого диапазона не включается — значит, разбиение действительно корректно и без пересечений.

Что именно нужно исправить в псевдокоде:

1. **Базовый случай (условие остановки рекурсии).** Условие `if l+1=r` `return`; останавливает рекурсию только когда в диапазоне ровно один элемент ($r - l = 1$). Но более аккуратно и надёжно писать условие как **`if r - l <= 1: return`** — это защищает алгоритм и от вырожденного случая, когда диапазон пустой ($r = l$, то есть $r - l = 0$), а не только от случая ровно одного элемента. Формально при корректном первом вызове (`mergeSort(a, 0, n)` с $n \geq 1$) пустой диапазон не должен возникать, но условие `<=1` — более безопасная и общепринятая формулировка, устойчивая к граничным случаям.
2. **Вызов Merge должен получать также середину mid.** В исходном псевдокоде вызывается `Merge(a, l, r)` — то есть *только* с границами всего диапазона, без информации о том, где именно проходит граница между двумя уже отсортированными половинами! А ведь функции `Merge` обязательно нужно знать, где заканчивается первая отсортированная половина и начинается вторая, чтобы правильно их слить. Поэтому корректный вызов — **`Merge(a, l, mid, r)`**, то есть с явной передачей середины.

Исправленный псевдокод:

```
mergeSort(a, l, r):
    if r - l <= 1:
        return                // 0 или 1 элемент — уже "отсортировано"
    mid = (l + r) / 2          // целочисленное деление
    mergeSort(a, l, mid)       // рекурсивно сортируем левую половину [l, mid)
    mergeSort(a, mid, r)       // рекурсивно сортируем правую половину [mid, r)
    Merge(a, l, mid, r)        // сливаем две отсортированные половины
```

Разбор функции Merge К моменту вызова `Merge(a, l, mid, r)` нам гарантированно известно, что: - подмассив `a[l..mid-1]` уже отсортирован (по возрастанию), - подмассив `a[mid..r-1]` тоже уже отсортирован.

Задача `Merge` — **слить** эти два отсортированных подмассива в один отсортированный подмассив `a[l..r-1]`, не потеряв ни одного элемента.

Идея слияния (по шагам):

1. Создаём **временный вспомогательный массив** `temp` размера $r - l$ (в него будем записывать результат слияния, потому что “перезаписывать на месте” исходный массив нельзя — можно затереть ещё не прочитанные элементы).
2. Заводим два указателя: $i = l$ (бежит по левой половине) и $j = mid$ (бежит по правой половине), а также указатель $k = 0$ для записи в `temp`.

3. Пока **оба** указателя i и j не вышли за пределы своих половин:
 - сравниваем $a[i]$ и $a[j]$;
 - **меньший** из двух элементов копируем в $temp[k]$, продвигаем соответствующий указатель (i или j) и k на единицу вперёд;
 - (если элементы равны — берём элемент из **левой** половины первым, это как раз и обеспечивает **устойчивость** сортировки слиянием, так как в исходном массиве элементы левой половины стояли раньше элементов правой).
4. Когда одна из половин “закончилась” (её указатель дошёл до конца), а в другой половине ещё остались элементы — **все оставшиеся элементы** этой второй половины **копируются в temp как есть** (они уже отсортированы между собой и, что важно, все они больше либо равны всем уже скопированным элементам — иначе они были бы скопированы раньше на шаге 3).
5. В конце — **копируем весь temp обратно** в исходный массив $a[l..r-1]$.

Пример слияния (иллюстрация идеи): пусть левая половина — $[2, 5, 8]$, правая — $[1, 6, 7]$. Сравниваем по одному элементу с “головы” каждой половины и каждый раз забираем меньший: 1 (из правой) $\rightarrow 2$ (из левой) $\rightarrow 5$ (из левой) $\rightarrow 6$ (из правой) $\rightarrow 7$ (из правой) $\rightarrow 8$ (из левой, остаток). Результат: $[1, 2, 5, 6, 7, 8]$ — отсортированный массив.

Сложность слияния: каждый элемент обоих подмассивов копируется в $temp$ ровно один раз, поэтому слияние диапазона размера $n = r - l$ выполняется за $O(n)$ (линейное время) и требует $O(n)$ дополнительной памяти под временный массив.

Оценка сложности сортировки слиянием Рекуррентное соотношение для времени работы (где n — размер сортируемого диапазона):

$$T(n) = 2T\left(\frac{n}{2}\right) + O(n),$$

где $2T(n/2)$ — время на рекурсивную сортировку двух половин, а $O(n)$ — время на последующее слияние. Решение такого рекуррентного соотношения (по построению дерева рекурсии: $\log n$ уровней, на каждом уровне суммарно выполняется $O(n)$ работы на слияния) даёт:

$$T(n) = O(n \log n).$$

Важная особенность: в отличие от сортировки вставками, у сортировки слиянием **лучший, худший и средний случаи асимптотически совпадают** — все они $\Theta(n \log n)$. Это происходит потому, что алгоритм всегда делит массив пополам и всегда выполняет полное слияние, **независимо** от того, в каком порядке изначально расположены элементы — количество сравнений может немного отличаться, но порядок роста не меняется.

Плата за скорость — память: сортировка слиянием (в стандартной реализации с временным массивом) требует $O(n)$ дополнительной памяти, в отличие от

сортировки вставками, которой достаточно $O(1)$. Это классический пример компромисса «время vs. память» при выборе алгоритма.

Устойчивость: сортировка слиянием **устойчива** — при условии, что на шаге сравнения в Merge при равенстве элементов мы всегда берём элемент из **левой** половины первым (как отмечено выше в пункте 3).

Краткое резюме связей между понятиями

- $T(n)$ — точная функция времени работы → мы упрощаем её до **порядка роста** через нотации O (сверху), Ω (снизу), Θ (точно).
 - Эти нотации **не связаны напрямую** с понятиями лучшего/худшего/среднего случая — это две разные оси классификации, которые комбинируются.
 - Классификация $O(1), O(\log n), \dots, O(n!)$ даёт шкалу «от эффективных к неприменимым на практике» алгоритмам.
 - Те же нотации применяются не только ко времени, но и к **памяти**.
 - При сортировке ключевую роль играют: **ключ сортировки** (что сравниваем) и **устойчивость** (сохраняется ли порядок одинаковых по ключу элементов) — это важно при последовательной сортировке по нескольким ключам.
 - **Сортировка вставками** — простой алгоритм $O(n)/O(n^2)/O(n^2)$ (лучший/средний/худший $O(1)$ памяти, устойчив).
 - **Сортировка слиянием** построена на методе «разделяй и властвуй», даёт стабильные $\Theta(n \log n)$ во всех случаях ценой $O(n)$ дополнительной памяти, устойчива при правильной реализации Merge.
-

6. Мастер-теорема (Master Theorem)

Выше мы получили рекуррентное соотношение для сортировки слиянием: $T(n) = 2T(n/2) + O(n)$, и «решили» его почти «на пальцах» (через дерево рекурсии), получив $T(n) = O(n \log n)$. **Мастер-теорема** — это общий инструмент, который позволяет решать *целый класс* подобных рекуррентных соотношений сразу, без построения дерева рекурсии каждый раз заново.

6.1. Постановка задачи

Многие алгоритмы, построенные по методу «разделяй и властвуй» (см. п. 5.4), приводят к рекуррентным соотношениям вида:

$$T(n) = aT\left(\frac{n}{b}\right) + f(n),$$

где:

- n — размер исходной задачи;

- $a \geq 1$ — **количество подзадач**, на которые разбивается исходная задача на каждом шаге рекурсии (сколько раз мы рекурсивно вызываем сами себя);
- $b > 1$ — **во сколько раз уменьшается размер** каждой подзадачи (то есть каждая из a подзадач имеет размер n/b);
- $f(n)$ — **стоимость «внешней» работы** на текущем уровне рекурсии, то есть всё то, что делается *помимо* самих рекурсивных вызовов: разбиение задачи на части (шаг «Разделяй») и объединение решений подзадач в решение исходной задачи (шаг «Объединяй»).

Пример. Для сортировки слиянием: $a = 2$ (два рекурсивных вызова), $b = 2$ (каждая половина в 2 раза меньше исходного массива), $f(n) = O(n)$ (столько стоит слияние Merge). То есть $T(n) = 2T(n/2) + O(n)$ — это ровно частный случай общей формулы выше.

Интуитивно, итоговая асимптотика $T(n)$ зависит от «борьбы» двух составляющих:

1. **Стоимости рекурсии** — насколько «дорого» само по себе разделение задачи на a подзадач размера n/b (эта часть, если её раскрутить полностью до тривиальных подзадач размера 1, растёт как $n^{\log_b a}$ — это ключевая величина, к которой всё сводится, см. доказательство ниже); 2. **Стоимости «внешней» работы** $f(n)$ на объединение/разбиение.

Мастер-теорема формализует, **какая из этих двух составляющих «побеждает»** асимптотически (или что они «сравнимы»), и в зависимости от этого даёт готовый ответ.

6.2. Формулировка

Пусть $T(n) = aT(n/b) + f(n)$, где $a \geq 1$, $b > 1$ — константы, а $f(n)$ — асимптотически положительная функция. Тогда:

Случай 1. Если $f(n) = O(n^{\log_b a - \varepsilon})$ для некоторой константы $\varepsilon > 0$ (то есть $f(n)$ растёт **строго медленнее**, чем $n^{\log_b a}$), то

$$T(n) = \Theta(n^{\log_b a}).$$

Случай 2. Если $f(n) = \Theta(n^{\log_b a})$ (то есть $f(n)$ растёт **с тем же порядком**, что и $n^{\log_b a}$), то

$$T(n) = \Theta(n^{\log_b a} \log n).$$

Случай 3. Если $f(n) = \Omega(n^{\log_b a + \varepsilon})$ для некоторой константы $\varepsilon > 0$ (то есть $f(n)$ растёт **строго быстрее**, чем $n^{\log_b a}$), **и** дополнительно выполняется **условие регулярности**:

$a f(n/b) \leq c f(n)$ для некоторой константы $c < 1$ и всех достаточно больших n ,

то

$$T(n) = \Theta(f(n)).$$

Смысл величины $n^{\log_b a}$ простыми словами: это «стоимость чистой рекурсии» — во сколько раз в итоге «размножилась бы» работа, если бы на каждом уровне рекурсии не делалось вообще никакой дополнительной работы, кроме самого деления задачи. Мастер-теорема, по сути, **сравнивает** функцию $f(n)$ (работу вне рекурсии) с этой «опорной» функцией $n^{\log_b a}$ и смотрит, кто растёт быстрее: - если «опорная» функция $n^{\log_b a}$ быстрее — она и определяет итоговую асимптотику (случай 1); - если обе функции растут одинаково — асимптотика та же, но с дополнительным множителем $\log n$, который возникает из-за «накопления» одинаковых по величине вкладов на каждом из $\log_b n$ уровней рекурсии (случай 2); - если $f(n)$ быстрее — именно она определяет итоговую асимптотику (случай 3), при условии, что она не «слишком неравномерно» растёт (это и обеспечивает условие регулярности).

Важное замечание, где легко ошибиться: мастер-теорема покрывает **не все** возможные соотношения между $f(n)$ и $n^{\log_b a}$. Между случаями 1–3 есть «зазоры» — например, если $f(n)$ растёт быстрее $n^{\log_b a}$, но не настолько, чтобы удовлетворить строгому неравенству с $\varepsilon > 0$ в случае 3 (то есть разница не полиномиальная, а, скажем, логарифмическая), теорема **не применима**, и нужно решать рекуррентное соотношение другими методами.

6.3. Доказательство (через дерево рекурсии)

Идея доказательства — представить процесс рекурсии в виде **дерева рекурсии** и просуммировать работу по всем уровням.

Построение дерева. Корень дерева соответствует исходному вызову с размером задачи n , в котором выполняется «внешняя» работа $f(n)$. У корня — a потомков, каждый из которых соответствует рекурсивному вызову с размером задачи n/b , и в каждом из них выполняется работа $f(n/b)$. У каждого из этих потомков — снова по a потомков размера n/b^2 , и так далее.

Уровни дерева. Рассмотрим уровень дерева с номером i (считая от корня, $i = 0, 1, 2, \dots$):

- **Количество узлов** на уровне i равно a^i (на каждом уровне число узлов увеличивается в a раз по сравнению с предыдущим, так как у каждого узла ровно a потомков).
- **Размер подзадачи** в каждом узле на уровне i равен n/b^i (на каждом уровне размер уменьшается в b раз).
- **Суммарная «внешняя» работа** на уровне i (то есть сумма $f(\cdot)$ по всем узлам этого уровня) равна:

$$a^i \cdot f\left(\frac{n}{b^i}\right).$$

Глубина дерева. Рекурсия останавливается, когда размер подзадачи становится равным константе (базовый случай, $O(1)$), то есть когда $n/b^i = 1$, откуда:

$$i = \log_b n.$$

Значит, дерево имеет $\log_b n + 1$ уровней (от уровня 0 до уровня $\log_b n$).

Число листьев. На последнем уровне ($i = \log_b n$) число узлов равно:

$$a^{\log_b n} = n^{\log_b a}$$

(это стандартное тождество для степеней: $a^{\log_b n} = n^{\log_b a}$ — его можно проверить, прологарифмировав обе части по основанию b). Каждый лист соответствует базовому случаю с постоянной стоимостью $\Theta(1)$, поэтому суммарная стоимость всех листьев есть $\Theta(n^{\log_b a})$ — именно эта величина и есть та самая «опорная функция», с которой мы сравниваем $f(n)$.

Суммарная стоимость всего дерева — это сумма по всем уровням от 0 до $\log_b n - 1$ (работа внутренних узлов) плюс стоимость листьев:

$$T(n) = \Theta(n^{\log_b a}) + \sum_{i=0}^{\log_b n - 1} a^i f\left(\frac{n}{b^i}\right).$$

Дальнейший анализ этой суммы и приводит к трём случаям теоремы:

- **Случай 1 ($f(n)$ растёт медленнее $n^{\log_b a}$):** можно показать, что при $f(n) = O(n^{\log_b a - \epsilon})$ сумма по уровням образует **убывающую геометрическую прогрессию** (каждый следующий уровень вносит меньше предыдущего в постоянное число раз), поэтому вся сумма ограничена сверху величиной, пропорциональной *последнему* (самому большому) слагаемому — то есть стоимости листьев $\Theta(n^{\log_b a})$. Значит, вся сумма, а с ней и всё дерево, имеет порядок $\Theta(n^{\log_b a})$, и вклад $f(n)$ асимптотически «не виден» на общем фоне.
- **Случай 2 ($f(n) = \Theta(n^{\log_b a})$):** тогда **каждый** из $\log_b n$ уровней вносит вклад одного и того же порядка $\Theta(n^{\log_b a})$ (можно проверить прямой подстановкой: $a^i f(n/b^i) = a^i \cdot \Theta((n/b^i)^{\log_b a}) = \Theta(n^{\log_b a})$ — то есть степень i полностью сокращается). Суммируя одинаковый вклад $\Theta(n^{\log_b a})$ по всем $\Theta(\log n)$ уровням, получаем суммарно $\Theta(n^{\log_b a} \log n)$.
- **Случай 3 ($f(n)$ растёт быстрее $n^{\log_b a}$, с условием регулярности):** здесь, наоборот, сумма по уровням образует **возрастающую** (геометрически) прогрессию — каждый следующий уровень (ближе к корню исходной задачи) вносит больше, чем предыдущий. Условие регулярности $af(n/b) \leq cf(n)$ как раз гарантирует, что эта прогрессия действительно убывает **в обратную сторону** достаточно быстро (т.е. вклад корня доминирует над суммой всех остальных уровней). Поэтому вся сумма ограничена (с точностью до констант) стоимостью **самого первого уровня** — то есть корня дерева, где работа равна $f(n)$. В сумме получаем $\Theta(f(n))$.

Таким образом, во всех трёх случаях итоговая асимптотика определяется **тем из двух источников работы (либо “размножение” рекурсии $n^{\log_b a}$, либо внешняя работа $f(n)$), который растёт быстрее**, а в пограничном случае, когда они растут одинаково, — добавляется множитель $\log n$ за счёт «накопления» одинаковых вкладов по всем уровням дерева.

6.4. Применение к сортировке слиянием

Проверим на уже знакомом примере. Для mergeSort: $a = 2, b = 2, f(n) = \Theta(n)$ (линейная стоимость Merge).

1. Вычисляем опорную величину: $n^{\log_b a} = n^{\log_2 2} = n^1 = n$.
2. Сравниваем $f(n)$ с этой величиной: $f(n) = \Theta(n) = \Theta(n^{\log_b a})$ — это в точности **случай 2** мастер-теоремы.
3. Следовательно:

$$T(n) = \Theta(n^{\log_2 2} \cdot \log n) = \Theta(n \log n).$$

Это в точности совпадает с результатом, который мы получили ранее «вручную» через дерево рекурсии для сортировки слиянием — мастер-теорема просто даёт этот результат мгновенно, без построения дерева заново, как только соотношение приведено к стандартному виду $T(n) = aT(n/b) + f(n)$.

Замечание о частой ошибке при применении теоремы. Нужно внимательно сравнивать именно **порядок роста** $f(n)$ с $n^{\log_b a}$, а не «на глаз» их внешний вид. Например, если бы $f(n)$ содержало дополнительный логарифмический множитель (скажем, $f(n) = \Theta(n \log n)$ при том же $a = 2, b = 2$, где $n^{\log_b a} = n$), то это уже была бы ситуация **между** случаями 2 и 3 (строго говоря, случай 3 не выполняется, так как нет полиномиального зазора n^ε) — и «базовая» мастер-теорема здесь **не применима напрямую**, требуется либо расширенная версия теоремы, либо иной метод решения рекуррентного соотношения.